Introduction: Prediction

PA4 - Prediction

Motivation

In the last assignment we explored what we can do if we find similar items in a database. But what else could we use to predict what a user might like? If a user has sufficient data, we could try to find which **other users** they are most similar to, using the same method that we used for song similarity. With this we can try to **predict** what rating a user would give a song that they **haven't rated** based on the rating of their most similar user.

Tasks

Tasks

Predict the missing ratings for songs that users did not rate. In PA1 we received our data as a CSV of song, user, rating. There is not always a rating for every song by every user; there can be gaps in the data. In this PA we will address those gaps.

- Calculate Similarity
- Predict Ratings

Calculate User Similarity

In PA3 we calculated similarity between songs. This time, we'll use the same exact method to calculate **similarity between users** and then **sort them from most similar to least similar**.

All the first steps stay the same: Clean up the data, normalize by user mean + SD. After this, we are going to flip the script. To compare two users to each other, find all songs that they both rated. So, in the simplest example, when we compare user1 to user2, they would only have both liked songX, resulting in a Euclidean distance of

distance = sqrt(x1 - x2)^2

We can use the same formula, just now x refers to the song and 1 and 2 refer to the user. This means that we need **at least two cooperative users** for this functionality! Throw an error if we have less than two cooperative users and gracefully exit the program.

Implementation Hint

By now you've learned about concepts like **Polymorphism** and **Interfaces**, and you're probably eager to try them out. This would be the perfect spot to do so as the core similarity calculation logic is the same whether it's comparing songs or users or anything

else for that matter. Consider creating an Interface SimilarityComparable (or something similar) that a Song class and a User class could implement. An interface like this could have methods for accessing normalized ratings and checking for the existence of a rating. So instead of having a method that takes **songs as the vectors** and **users as dimensions** (in the Euclidean Distance calculation) and calculates song similarity, consider changing it into a method that takes any list of SimilarityComparable and any list of dimensions and calculates similarity between them.

This **eliminates code duplication** and makes your code flexible for possible extensions. Of course it's not the only way to do this, but it works as an elegant simplification to your code, so you can save time on the next PAs.

Predict Missing Ratings

To predict ratings for a target user who hasn't rated a specific song, we **borrow the normalized rating from the most similar other user who has rated that song**, then de-normalize it using the target user's mean and standard deviation. If the **most similar** other user has **not rated** the song, then you look at the **next most similar** user and so on until you find a normalized rating to borrow. If all options are exhausted and there is no similar user who has rated the song, the predicted rating will be set to NaN (see section "Edge cases").

predictedRating = round(otherUser.normalizedRating * targetUser.stdDeviation + targetUser.mean)

Reminder a **smaller** similarity score means two users are **similar**, while a **larger** score represents a **larger difference** between two users' rating tendencies.

Edge cases

There are two edge cases to account for when predicting the missing ratings:

- **No similar user who rated the song:**
  If there is no other user who both (a) has a valid similarity score with the target user and (b) has rated the song, the predicted rating will be set to NaN.
  Example: user5 has not rated song2, and the only users who rated song2 (user1, user2) have no similarity score with user5. Result: user5, song2 → NaN.
- **Predicted rating outside the allowed range:**
  If the de-normalized predicted rating is less than 1, set it to 1. If it is greater than 5, set it to 5.

Example

Example:

song1,user1,2

song2,user1,3

song4,user1,4

song1,user2,2

song2,user2,3

song4,user2,5

song1,user3,4

song3,user3,5

song4,user3,4

song5,user3,5

song1,user4,1

song3,user4,2

song4,user4,1

song5,user5,3

song6,user5,2

Raw Ratings

	user1	**user2**	**user3**	**user4**	**user5**
song1	2	2	4	1	-
song2	3	3	-	-	-
song3	-	-	5	2	-

song4	4	5	4	1	-
song5	-	-	5	-	3
song6	-	-	-	-	2

Normalized Ratings

	user1	**user2**	**user3**	**user4**
song1	-1.224744871391589	-1.0690449676496978	-1.0	-0.707106781186
song2	0.0	-0.2672612419124245	-	-
song3	-	-	1.0	1.414213562373
song4	1.224744871391589	1.3363062095621219	-1.0	-0.707106781186
song5	-	-	1.0	-
song6	-	-	-	-

User Similarity

-----	**user1**	**user2**	**user3**

user1	0.0	0.32881144084141245	2.23606797749979
user2	0.32881144084141245	0.0	2.337326231486798
user3	2.23606797749979	2.337326231486798	0.0
user4	1.9999999999999998	2.0752194827489103	0.5857864376269052
user5	NaN	NaN	0.0

user -> other users in most similar order left to right

user1: [user2, user4, user3]

user2: [user1, user4, user3]

user3: [user5, user4, user1, user2]

user4: [user3, user1, user2]

user5: [user3]

Normalized Ratings With Predictions

	user1	**user2**	**user3**	**user4**
song1	-1.224744871391589	-1.0690449676496978	-1.0	-0.70710678118
song2	0.0	-0.2672612419124245	***0.0***	***0.0***
song3	***1.4142135623730951***	***1.4142135623730951***	1.0	1.414213562373

song4	1.224744871391589	1.3363062095621219	-1.0	-0.70710678118
song5	***1.0***	***1.0***	1.0	***1.0***
song6	-	-	***-1.0***	-

Raw Ratings With Predictions

	user1	**user2**	**user3**	**user4**	**user5**
song1	2	2	4	1	***2***
song2	3	3	***5***	***1***	***NaN***
song3	***4***	***5***	5	2	***3***
song4	4	5	4	1	***2***
song5	***4***	***5***	5	***2***	3
song6	***NaN***	***NaN***	***4***	***NaN***	2

Output

Calculate a similarity table of all of the users and then predict the missing ratings. The output file will contain a CSV header at the top in the format of song,user,predicted rating. Each row will contain the predicted raw ratings of the missing ratings of users in lexicographical order by song and then name. Denormalize the predicted value for the user and **round** it to the nearest integer.

Error Handling

Error Handling

- All previous error cases apply, and all similarity (PA3) errors apply to prediction (PA4).
- If there are less than two cooperative users, we can't calculate user similarities, so treat this as an error!
- If there are **no missing ratings** (every song has been rated by every user) and the -p arg is passed, you should **treat that as an error**.
- If a third argument **other than** -a, -u or -p is present, then you should treat that as an error.

Running Your Program

Running Your Program

There are now **three different ways** to run your program. The first argument is the input CSV file and the second argument is the output CSV file.

- Just as in PA1 and PA2, when two arguments are passed to your program, you will write the song rating stats to a CSV file corresponding to the second arg.
- When a third argument -u is passed to your program, then you will write the user similarities to a CSV file corresponding to the second arg.
- When a third argument -p is passed to your program, you will write the user predictions to a CSV file corresponding to the second arg.

These are independent from one another, so **only one file** is created **in either case**, only the contents of the output file changes depending on the presence of the third arg -u or -p. To accept command line arguments, utilize the String[] args parameter in your main method.

Do not hard code the input or output files. The user of your program must be able to specify which input CSV they want to read from and where and what name they want for the output CSV.

Example Input and Output

Example Input/Output

command

gradle run -q --args="'database/files/file2.csv' 'output.csv'"

output.csv

song,number of ratings,mean,standard deviation

song1,2,2.5,1.5

song2,2,3.5,1.5

song3,1,1.0,0.0

command

gradle run -q --args="'database/files/file1.csv' 'similarity.csv' '-u'"

similarity.csv

name1,name2,similarity

user1,user2,0.0

user1,user3,2.8284271247461903

user2,user3,2.8284271247461903

command

gradle run -q --args="'database/files/file4.csv' 'user_predictions.csv' '-p'"

user_predictions.csv

song,user,predicted rating

All Star,charlie,4

Bohemian Rhapsody,alex,2

Sweet Home Alabama,alex,3

We Will Rock You,cameron,1

We Will Rock You,charlie,NaN

command

gradle run -q --args="'database/files/file3.csv' 'existing_folder/predictions.csv' '-p'"

existing_folder/predictions.csv

song,user,predicted rating

song1,user5,2

song2,user3,5

song2,user4,1

song2,user5,NaN

song3,user1,4

song3,user2,5

song3,user5,3

song4,user5,2

song5,user1,4

song5,user2,5

song5,user4,2

song6,user1,NaN

song6,user2,NaN

song6,user3,4

song6,user4,NaN

command

gradle run -q --args="'database/files/file1.csv' 'output.csv' '-p'"

System.err

Error: no predictions to be made

Hints

Hints

- The predicted rating must be rounded to the nearest integer. Keep in mind the differences between rounding and truncating.
- Test and develop your program on the department Linux machines. That is where we will evaluate it. Your grade depends on how your program performs on those machines. (Warning: Java may behave differently across different environments.)